

ASSIGNMENT – 15.3

AI ASSISTED CODING

NAME : L.NAVYA

HALLTICKET NO : 2403A51308

BATCH NUMBER : 01

COURSE CODE : 24CS002PC215

PROGRAM NAME : B.TECH

YEAR/SEM : 2nd AND 3RD

TASK 1 :

Ask AI to generate a Flask REST API with one
route:GET

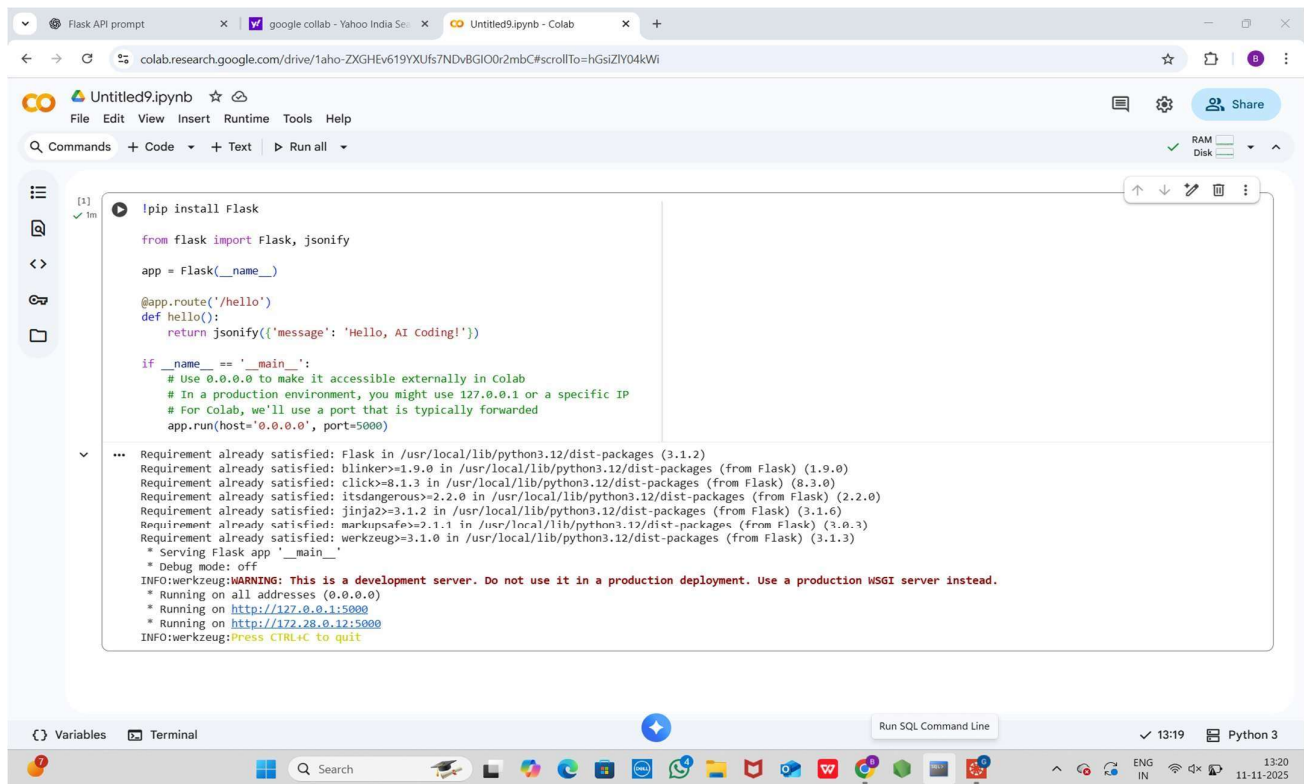
/hello → returns {"message": "Hello, AI Coding!"}

PROMPT :

“Create a Flask REST API with one route: GET /hello → returns

JSON { 'message': 'Hello, AI Coding!' }”

CODE :



The screenshot shows a Google Colab notebook titled 'Untitled9.ipynb'. The code cell contains the following Python code:

```
[1] ✓ 1m
!pip install Flask

from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/hello')
def hello():
    return jsonify({'message': 'Hello, AI Coding!'})

if __name__ == '__main__':
    # Use 0.0.0.0 to make it accessible externally in Colab
    # In a production environment, you might use 127.0.0.1 or a specific IP
    # For Colab, we'll use a port that is typically forwarded
    app.run(host='0.0.0.0', port=5000)
```

The output of the code cell shows the following messages:

```
... Requirement already satisfied: Flask in /usr/local/lib/python3.12/dist-packages (3.1.2)
Requirement already satisfied: blinker>=1.9.0 in /usr/local/lib/python3.12/dist-packages (from Flask) (1.9.0)
Requirement already satisfied: click>=8.1.3 in /usr/local/lib/python3.12/dist-packages (from Flask) (8.3.0)
Requirement already satisfied: itsdangerous>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from Flask) (2.2.0)
Requirement already satisfied: Jinja2>=3.1.2 in /usr/local/lib/python3.12/dist-packages (from Flask) (3.1.6)
Requirement already satisfied: MarkupSafe>=2.1.1 in /usr/local/lib/python3.12/dist-packages (from Flask) (3.0.3)
Requirement already satisfied: Werkzeug>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from Flask) (3.1.3)
* Serving Flask app '__main__'
* Debug mode: off
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.28.0.12:5000
INFO:werkzeug:Press CTRL+C to quit
```

The bottom of the screenshot shows the Windows taskbar with various application icons and the system clock displaying 13:20 on 11-11-2025.

OUTPUT :

* Running on all addresses (0.0.0.0)

* Running on [h p://127.0.0.1:5000](http://127.0.0.1:5000)

* Running on [h p://172.28.0.12:5000](http://172.28.0.12:5000)

INFO:werkzeug:Press CTRL+C to quit

TASK 2 :

Use AI to build REST endpoints for a Student API:

- GET /students → List all students.
- POST /students → Add a new student.
- PUT /students/<id> → Update student details.
- DELETE /students/<id> → Delete a student.

PROMPT :

“Create a Flask REST API for Students with CRUD: GET /students, POST /students, PUT /students/<id>, DELETE /students/<id>. Use list/dictionary storage and return JSON.”

CODE :

The image displays two screenshots of a Google Colab notebook. The top screenshot shows a code cell with a Flask application for managing a list of students. The code includes imports for request and jsonify, in-memory storage for students, and three routes: GET /students, POST /students, and PUT /students/<int:student_id>. The bottom screenshot shows the output of the first cell, which includes the command 'pip install Flask' and the execution of the Flask application. The output shows that all required packages are already satisfied and that the application is running on http://172.28.0.12:5000.

```
from flask import request, jsonify

# In-memory storage for students
students = []
student_id_counter = 1

@app.route('/students', methods=['GET'])
def get_students():
    return jsonify(students)

@app.route('/students', methods=['POST'])
def add_student():
    global student_id_counter
    new_student = request.json
    if 'name' not in new_student:
        return jsonify({'error': 'Student name is required'}), 400
    new_student['id'] = student_id_counter
    students.append(new_student)
    student_id_counter += 1
    return jsonify(new_student), 201

@app.route('/students/<int:student_id>', methods=['PUT'])
def update_student(student_id):
    student_to_update = None
    for student in students:
        if student['id'] == student_id:
            student_to_update = student
            break

    if student_to_update:
        update_data = request.json
        student_to_update.update(update_data)
        return jsonify(student_to_update)
    return jsonify({'error': 'Student not found'}), 404
```

```
!pip install Flask

from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/hello')
def hello():
    return jsonify({'message': 'Hello, AI Coding!'})

if __name__ == '__main__':
    # Use 0.0.0.0 to make it accessible externally in Colab
    # In a production environment, you might use 127.0.0.1 or a specific IP
    # For colab, we'll use a port that is typically forwarded
    app.run(host='0.0.0.0', port=5000)
```

```
Requirement already satisfied: Flask in /usr/local/lib/python3.12/dist-packages (3.1.2)
Requirement already satisfied: blinker>=1.9.0 in /usr/local/lib/python3.12/dist-packages (from Flask) (1.9.0)
Requirement already satisfied: click>=8.1.3 in /usr/local/lib/python3.12/dist-packages (from Flask) (8.3.0)
Requirement already satisfied: itsdangerous>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from Flask) (2.2.0)
Requirement already satisfied: Jinja2>=3.1.2 in /usr/local/lib/python3.12/dist-packages (from Flask) (3.1.6)
Requirement already satisfied: MarkupSafe>=2.1.1 in /usr/local/lib/python3.12/dist-packages (from Flask) (3.0.3)
Requirement already satisfied: Werkzeug>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from Flask) (3.1.3)
* Serving Flask app '__main__'
* Debug mode: off
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://172.28.0.12:5000
* Running on http://172.28.0.12:5000
INFO:werkzeug:Press CTRL+C to quit
```

OUTPUT :

The app.run() call is in the first cell (hGsiZlY04kWi), so we don't need it here.

if __name__ == '__main__':

```
# app.run(host='0.0.0.0', port=5000)
```

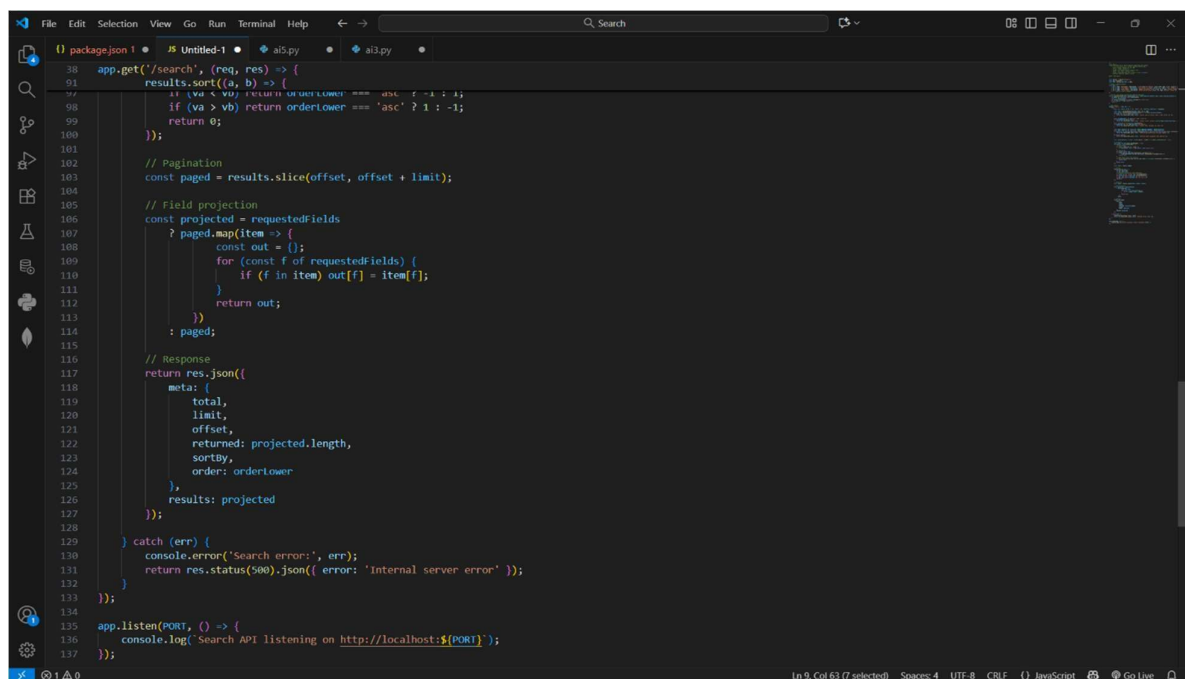
TASK 3 :

Ask AI to generate a REST API endpoint

PROMT :

“Create a REST API endpoint with query parameters for searching data. Include proper query param handling and return results in JSON.”

CODE :

A screenshot of a Visual Studio Code editor window. The editor is open to a file named 'ai3.py'. The code is a JavaScript/Node.js script implementing a REST API endpoint. It uses the 'express' framework. The script defines a 'GET' route for '/search'. It handles query parameters 'q', 'limit', 'offset', 'sortby', and 'order'. It uses 'mongoose' to query a database. The results are paginated and projected. The response is a JSON object with 'meta' (total, limit, offset, returned, sortby, order) and 'results'. It includes error handling for search errors and internal server errors. The script also listens on a specified port (5000) and logs the listening status.

```
108 app.get('/search', (req, res) => {
109   results.sort((a, b) => {
110     if (va < vb) return orderLower === 'asc' ? 1 : -1;
111     if (va > vb) return orderLower === 'asc' ? 1 : -1;
112     return 0;
113   });
114   // Pagination
115   const paged = results.slice(offset, offset + limit);
116   // Field projection
117   const projected = requestedFields
118     ? paged.map(item => {
119       const out = {};
120       for (const f of requestedFields) {
121         if (f in item) out[f] = item[f];
122       }
123       return out;
124     })
125     : paged;
126   // Response
127   return res.json({
128     meta: {
129       total,
130       limit,
131       offset,
132       returned: projected.length,
133       sortby,
134       order: orderLower
135     },
136     results: projected
137   });
138 } catch (err) {
139   console.error('Search error:', err);
140   return res.status(500).json({ error: 'Internal server error' });
141 }
142 });
143
144 app.listen(PORT, () => {
145   console.log('Search API listening on http://localhost:${PORT}');
146 });
```

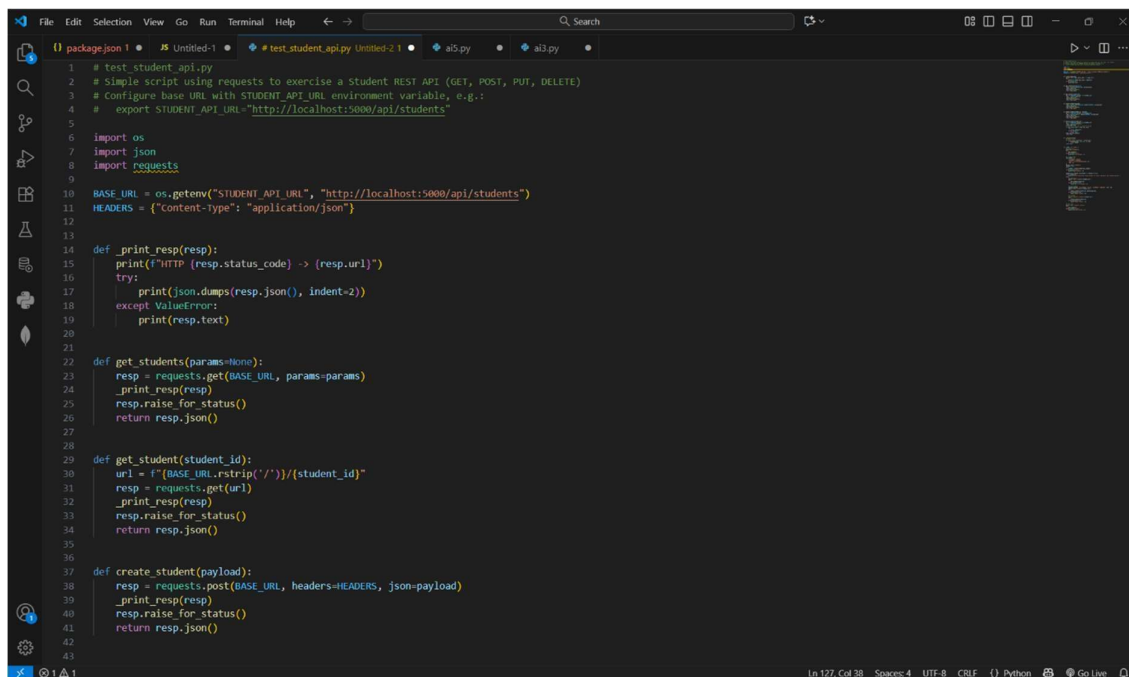
TASK 4 :

Ask AI to write test scripts using Python requests module to call .APIs created above.

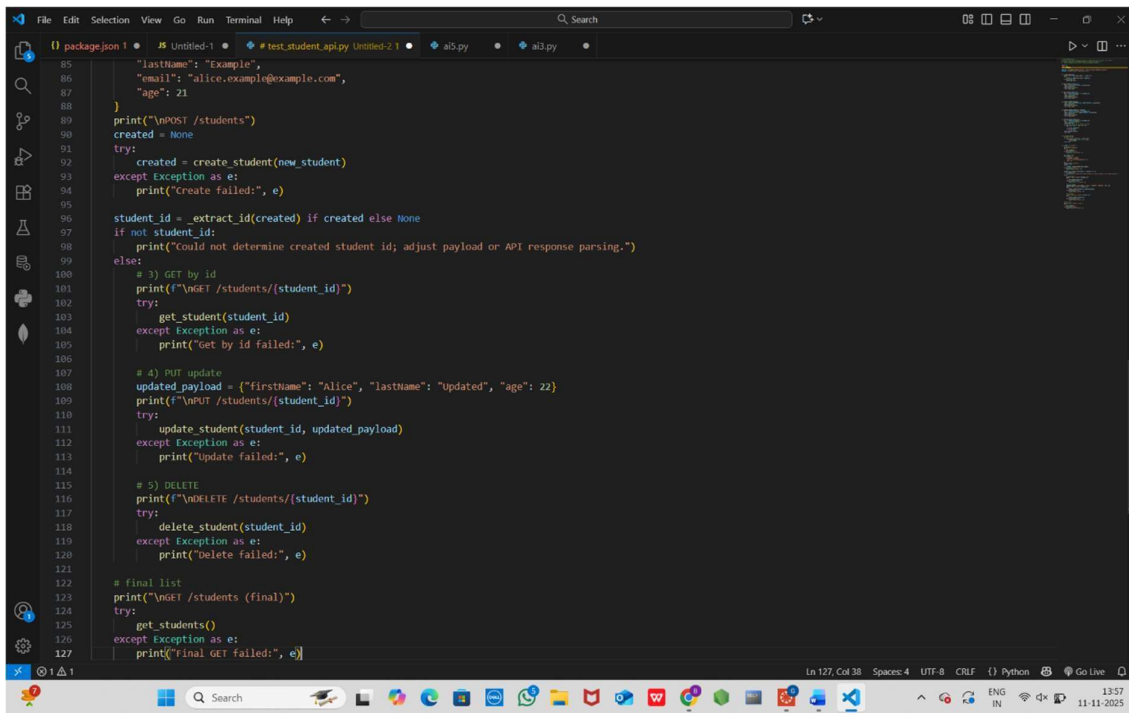
PROMPT :

“Write Python test scripts using the requests module to call REST API endpoints (GET, POST, PUT, DELETE) for the Student API, and print the JSON responses.”

CODE :

A screenshot of a Visual Studio Code editor window. The editor has a dark theme and shows a Python file named 'test_student_api.py'. The code is a script for testing a REST API using the 'requests' module. It includes comments, imports for 'os', 'json', and 'requests', and defines three functions: '_print_resp', 'get_students', and 'create_student'. The script is currently at line 127, column 38. The status bar at the bottom indicates the file is in UTF-8 encoding with CRLF line endings, and it's a Python file. The 'Go Live' button is visible in the bottom right corner.

```
1 # test_student_api.py
2 # Simple script using requests to exercise a Student REST API (GET, POST, PUT, DELETE)
3 # Configure base URL with STUDENT_API_URL environment variable, e.g.:
4 # export STUDENT_API_URL="http://localhost:5000/api/students"
5
6 import os
7 import json
8 import requests
9
10 BASE_URL = os.getenv("STUDENT_API_URL", "http://localhost:5000/api/students")
11 HEADERS = {"content-type": "application/json"}
12
13
14 def _print_resp(resp):
15     print(f"HTTP {resp.status_code} -> {resp.url}")
16     try:
17         print(json.dumps(resp.json(), indent=2))
18     except ValueError:
19         print(resp.text)
20
21
22 def get_students(params=None):
23     resp = requests.get(BASE_URL, params=params)
24     _print_resp(resp)
25     resp.raise_for_status()
26     return resp.json()
27
28
29 def get_student(student_id):
30     url = f"{BASE_URL.rstrip('/')}/{student_id}"
31     resp = requests.get(url)
32     _print_resp(resp)
33     resp.raise_for_status()
34     return resp.json()
35
36
37 def create_student(payload):
38     resp = requests.post(BASE_URL, headers=HEADERS, json=payload)
39     _print_resp(resp)
40     resp.raise_for_status()
41     return resp.json()
42
43
44
```



```
85     "lastName": "Example",
86     "email": "alice.example@example.com",
87     "age": 21
88 }
89 print("\nPOST /students")
90 created = None
91 try:
92     created = create_student(new_student)
93 except Exception as e:
94     print("Create failed:", e)
95
96 student_id = _extract_id(created) if created else None
97 if not student_id:
98     print("Could not determine created student id; adjust payload or API response parsing.")
99 else:
100     # 3) GET by id
101     print(f"\nGET /students/{student_id}")
102     try:
103         get_student(student_id)
104     except Exception as e:
105         print("get by id failed:", e)
106
107     # 4) PUT update
108     updated_payload = {"firstName": "Alice", "lastName": "Updated", "age": 22}
109     print(f"\nPUT /students/{student_id}")
110     try:
111         update_student(student_id, updated_payload)
112     except Exception as e:
113         print("Update failed:", e)
114
115     # 5) DELETE
116     print(f"\nDELETE /students/{student_id}")
117     try:
118         delete_student(student_id)
119     except Exception as e:
120         print("Delete failed:", e)
121
122 # final list
123 print("\nGET /students (final)")
124 try:
125     get_students()
126 except Exception as e:
127     print("final GET failed:", e)
```

OUTPUT :

* Serving Flask app '__main__'

* Debug mode: off